

LLD Chapter 0: The 4-Step Lego Blueprint & Pattern Decision Tree

Core Learning Objective: Transform Low-Level Design from an intimidating, open-ended puzzle into a calm, mechanical, 4-step assembly process. Master the visual Pattern Decision Tree to select the perfect GoF pattern for any interview question in seconds.

1. The 4-Step "Lego" LLD Problem-Solving Blueprint

Every Low-Level Design problem (Parking Lot, Elevator, Movie Booking, Rate Limiter) is solved using the exact same 4 mechanical steps:

flowchart TD

Step1["Step 1: Extract the Nouns → Entity Classes
(Read problem, list all physical entities)"]

Step2["Step 2: Extract the Verbs → Public Methods
(What actions do users or systems perform?)"]

Step3["Step 3: Identify What Changes → Design Patterns
(Do algorithms vary? → Strategy, Factory, Singleton)"]

Step4["Step 4: Add Concurrency Defense → Thread Locks
(Identify shared mutable state)"]

Step1 → Step2 → Step3 → Step4

Walkthrough on a Real Example: "Design a Parking Lot"

1. **Nouns:** Vehicle, ParkingSpot, Ticket, Payment, Floor. $\rightarrow$ *These are your classes!*
2. **Verbs:** Park, Unpark, IssueTicket, CalculateFee. $\rightarrow$ *These are your class methods!*
3. **What Changes:**
 - Spot allocation changes (Nearest vs Lowest Floor). $\rightarrow$ *Use Strategy Pattern!*
 - Vehicle types differ (Car, Bike, Truck). $\rightarrow$ *Use Factory Pattern!*
 - The parking lot configuration is a single physical facility. $\rightarrow$ *Use Singleton Pattern!*
4. **Concurrency:** Two cars attempting to grab the same spot simultaneously. $\rightarrow$ *Wrap `ParkingSpot.park()` in a `threading.Lock()`!*

That's it! In under 3 minutes, your entire architecture is completely solved before typing a line of code.

2. The Visual Pattern Decision Tree ("Which Pattern Do I Use When?")

flowchart TD

Q1{"What is the core architectural challenge?"}

Q1 → |Object Creation is Complex or Dynamic| Creational{"How does creation vary?"}

Creational → |By Type or Parameter| Factory["Factory Pattern / Abstract Factory"]

Creational → |Dozens of Optional Steps/Fields| Builder["Builder Pattern"]

Creational → |Only 1 Global Instance Permitted| Singleton["Thread-Safe Singleton Pattern"]

Q1 → |Different Interfaces or Added Behaviors| Structural{"What is the relationship?"}

Structural → |Incompatible Third-Party Interface| Adapter["Adapter Pattern"]

Structural → |Add Responsibilities Dynamically| Decorator["Decorator Pattern"]

Structural → |Lazy Loading or Security Caching| Proxy["Proxy Pattern"]

Q1 → |Algorithms, State or Object Communication| Behavioral{"What is changing dynamically?"}

Behavioral → |Algorithm or Calculation Strategy| Strategy["Strategy Pattern"]

Behavioral → |Object behavior changes as state changes| State["State Pattern"]

Behavioral → |One object changes; notify multiple others| Observer["Observer Pattern"]

Behavioral → |Encapsulate request as object (Queue/Undo)| Command["Command Pattern"]

3. The 14 Core Systems Rosetta Stone Matrix

Click on any system to jump directly into its complete, runnable Python implementation with full class hierarchies, design patterns, and thread-safety tests:

System	Primary Pattern	Secondary Pattern	The Critical Concurrency Trap
1. Distributed Job Scheduler	Command (Jobs)	Strategy (FCFS / Priority)	Worker race on queue; Lock during dispatch
2. Library Management	State (Available/Reserved)	Observer (Hold notification)	Race on last available physical book copy
3. Movie Ticket Booking	State Machine (Held/Booked)	Strategy (Payment)	Critical: 1,000 users booking seat A1; fine-grained mutex + TTL
4. Car Rental System	Factory (Vehicle types)	Strategy (Daily / Surge fare)	Inter-branch vehicle availability sync
5. Parking Lot	Singleton (Facility)	Strategy (Spot allocation)	Dual cars entering gate grabbing same spot
6. Inventory Management	Observer (Low stock alerts)	Strategy (Fulfillment routing)	Overselling stock during concurrent orders
7. Ride Sharing (Uber)	Strategy (Matching / Surge)	State (Trip lifecycle)	Two drivers accepting the same ride request
8. Rate Limiter	Strategy (Token / Leaky bucket)	Encapsulation (Atomic counters)	Atomic token refill under 50,000 QPS
9. Snake and Ladders	State (Player turn queue)	Strategy (Dice rolling)	Edge overshoot past 100 win condition
10. Elevator System	State (Up / Down / Idle)	Strategy (LOOK / SCAN algorithm)	Race on hall call registration
11. Vending Machine	State (Idle / Coin / Dispense)	State Transitions	Dispensing item without sufficient coin balance
12. In-Memory File System	Composite (Files & Dirs)	Trie (Path traversal)	Concurrency lock on parent directory path mutations
13. High-Throughput Logger	Singleton + Observer	RingBuffer (Zero-lock queue)	Producer-consumer backpressure & buffer overflow
14. Pub-Sub Message Broker	Observer (Subscribers)	Strategy (Partitioning)	Fan-out race conditions & consumer offset tracking